

Coding standards

1.Introduction

This doc provides coding standards and guidelines used by Omnitech to ensure quality, maintainability, and consistency for the driving tracker project.

General guidelines

Write clean, readable and maintainable code. Follow KISS(Keep it Simple, Stupid) principles. Avoid over-engineering solutions when a simpler implementation suffices.

Naming Conventions

The naming conventions used throughout the project are specified below. These standards are used to ensure that code is readable and maintainable across the entire project

3.1 Backend

Through out the entire backend of the project the convention used is `snake_case`.

3.1.1 Variables

- Variables in the backend are given meaningful and descriptive names
Good example:

```
user_id: string ;
```

Bad example:

```
ud: string;
```

3.1.2 Functions

- Functions are given descriptive names that clearly state their action.

```
async get_all_vehicles(data:get_vehicles): Promise<any[]> {
    const user_id = data.user_id;
    if(!user_id){
        throw new Error("Missing field(s)");
    }
    // the rest of the code
}
```

3.2 Frontend (Application)

- The frontend is Kotlin based where camelCase is used for functions and variables following Kotlin standards

3.2.1 Variables

- Variables are given meaningful and descriptive names in camelCase
Good example:

```
subscriptionKey: String
```

Bad example:

```
sKey: String
```

3.2.2 Constants

- Constants in Kotlin (especially inside companion objects) should use UPPER_SNAKE_CASE
Good example:

```
private val STOP_SPEED_THRESHOLD_KMH = 5f
```

Bad example:

```
private val StopSpeedThresholdKmh = 5f
```

3.2.3 Functions and Classes

- Use of descriptive names to understand what the function is meant to do.

```
class TripSummaryViewModel @Inject constructor(private val
```

```
suspend fun getVehicles(): Result<List<VehicleDto>>
```

4. Commenting

- Use of inline comments sparingly and only for complex or non-obvious code blocks
- While the use of block comments is used to explain code sections

```
// Otherwise try to convert to number  
return Number(value);
```

```
export const contact_services = {  
  /**  
   * Creates a trusted contact row for owner user_id by loo  
   *  
   * identifier can be username OR user_id
```

```
* disallow adding yourself
* disallow duplicates
*/
```

5.Code Structure

5.1 Indentation

- The indentation used throughout the project is standard indentation of four spaces (via tab) and two spaces in the html and javascript components of the app

```
export const trips_services ={
  async create(data: create_trip){
    if(!data.user_id || !data.vehicle_id ){
      throw new Error("Missing required fields");
    }
  }
}
```

5.2 File structure

- The backend file structure splits services, routes and controllers
- The Android side file structure splits into data, di (dependency injection), services, ui and utilities

```
Driving-Tracker/
  android-app/
  backend/
  docs/
  frontend/
```

```
backend/
  src/
    services/
    routes/
```

```
controllers/
```

```
android-app//  
  data/  
  di/  
  services/  
  ui/  
  utils/
```

6. Testing

- Unit tests are written to check the controller and service levels of the api. New tests should be written for every new endpoint added
- Integration tests are being used to test the endpoints in full including controller and service logic
- End-to-End tests are used to test use cases simulating a user stepping through the android application and performing certain actions

7. Error Handling

7.1 Backend

- Services and utility functions throw errors that are caught in the controller layer, and formatted into an error response object

```
//service  
try{  
  //where the creation of the trip will happen  
  const user = await prisma.users.findUnique({  
    where: {user_id: data.user_id}  
  });  
  if(!user){  
    throw new Error("user not found");  
  }  
}
```

```

        }

    }
    //rest of code

//controller

catch(error: any){

    if (error.message.includes("user not found")){
        return res.status(403).json({
            error: "USER_NOT_FOUND",
            message: "User not found"
        });
    }
    //rest of code

```

7.2 Frontend (Application)

- try-catch blocks should be utilized around code blocks that could throw errors, such as network calls

```

return try {
    val response = api.searchAddress(query)
    Result.success(response.data) // Return the 1
} catch (e: HttpException) {
    val error = ApiErrorParser.parse(e)
    Result.failure(ApiException(error.error, error.message))
} catch (e: Exception) {
    Result.failure(ApiException("NETWORK_ERROR", e.message))
}
//rest of code

```

- Errors affecting specific UI components should be managed via scoped stateFlow variables

```
private val _tripStartState = MutableStateFlow<UiState>(U
val tripStartState: StateFlow<UiState> = _tripStartState

.
.
.

_tripStartState.value = UiState.Error(
                                code = exception.errorCod
                                message = exception.error
                                )
```

8.Code Review

- Our code is screened during each PR via sonarCube as well as codecov for test coverage to ensure that code maintains a good standard
- At least one review is required from a teammate to merge a PR, to maintain accountability

5.4 Indentation

- The indentation used throughout the project is standard indentation of four spaces (via tab) and two spaces in the html and javascript components of the app

```
export const trips_services ={
  async create(data: create_trip){
    if(!data.user_id || !data.vehicle_id ){
      throw new Error("Missing required fields");
    }
    //rest of code
```

```
fun loadApprovedContacts() {
```

```
viewModelScope.launch{
    _approvedContactsState.value = UiState.Loading

    contactsRepository.fetchApprovedContacts().for
        onSuccess = {contacts ->
            _approvedContactsState.value = UiStat

            //rest of code
        }
}
```